

Machine Learning Project Report

Yuejia Bai
Student ID: s4985919
Semester 1, 2026

Abstract

This report applies several machine learning models to the Australian Food Composition Database nutrient profile dataset. The main aim is to test whether the numerical nutrient composition of a food item can predict its broad food classification group. The task is treated as a multi-class classification problem. I cleaned the dataset, created a target variable from the broad food classification code, removed very small classes, handled missing values using median imputation, and compared a dummy baseline, k-nearest neighbours, logistic regression, a decision tree and a random forest classifier. The random forest model performed best, with a test balanced accuracy of 0.974 and a test macro F1 score of 0.976. This revised version puts more focus on justifying preprocessing choices, explaining which columns were removed and why, using feature importance as an algorithmic feature-selection check, and discussing why PCA was considered but not used as the final representation. The results suggest that broad food groups can be predicted very well from nutrient profiles, but the high performance should be interpreted carefully because food groups are naturally related to nutrient composition.

1 Introduction

The dataset used for this assignment contains nutrient composition information for a range of foods from the Australian Food Composition Database. The assignment is open-ended, so I chose a classification task that is directly related to the dataset: predicting the broad food classification group of a food item using only its nutrient values. My main research question is:

Can the nutrient profile of a food item be used to predict its major food classification group?

This question is suitable for machine learning because each food item has many numerical features. These features include macronutrients, minerals, vitamins, fatty acids and other nutrition-related values. It is reasonable to expect that the nutrient profile contains information about the type of food. For example, foods in cereal groups may have more carbohydrate and starch, meat or fish groups may have more protein, and oils or spreads may have high fat-related values. At the same time, the task is still useful because different food groups can overlap. Processed foods, mixed dishes, and foods with similar nutrient profiles may be harder to separate.

The main purpose of this report is not just to get a high score. The course assignment asks for a clear analysis process, results and discussion. Therefore, I focus on showing the complete machine learning workflow: defining the target, cleaning the data, choosing features, splitting the data, comparing models, evaluating the final model and interpreting what the model has learned. I also include a reflection on what I learned while doing the experiment.

I compared five models: a dummy baseline classifier, k-nearest neighbours, logistic regression, a single decision tree and a random forest classifier. These models were chosen because they represent different levels of complexity. The dummy classifier gives a baseline. kNN is simple and distance-based. Logistic regression is a linear model. Decision trees are non-linear and interpretable. Random forests are ensemble models that combine many trees and often perform well on tabular data. I included both a decision tree and a random forest because

this lets me compare one interpretable tree with an ensemble tree method. This is useful for showing the difference between a simple non-linear model and a more stable ensemble model.

2 Dataset and Problem Definition

The data file used in this report is AFCD Release 3 - Nutrient profiles.xlsx. I used the nutrient profile table for solids and liquids per 100 g. Each row represents a food item and the columns include food identification information, food classification information, nutrient values and some derivation information.

The prediction target was created from the food classification code. I used the first two digits of the classification code as the broad major food group. This means the model predicts a general food group rather than a very specific product. I made this choice because predicting very detailed categories would make the task less stable, especially when some categories have only a few examples. Predicting broad groups is also easier to explain and more suitable for this assignment.

I made one important feature decision: I did not use food names as model inputs. If the model used names such as bread, milk, beef or apple, it could learn from words instead of nutrient patterns. That would make the task unrealistically easy and would not answer the question I wanted to investigate. For the same reason, I removed administrative columns and derivation columns from the model features. The final input features were numerical nutrient columns only.

After constructing the broad target variable, I removed groups with fewer than 50 rows. I did this because extremely small classes can make training and evaluation unstable. For example, a class with only a few examples could be missing from a test split or could give a very noisy recall value. This filtering step left 1,253 rows, 90 nutrient features and 8 broad food groups. The selected groups had the following sample counts: group 18 had 355 rows, group 24 had 252 rows, group 12 had 163 rows, group 16 had 143 rows, group 15 had 115 rows, group 13 had 112 rows, group 11 had 57 rows and group 19 had 56 rows.

The final task is a multi-class classification problem. The input is a vector of nutrient values for one food item. The output is one of the selected major food groups.

3 Preprocessing

The preprocessing stage was important because the raw spreadsheet was not immediately ready for machine learning. I used the following process:

1. Load the nutrient profile spreadsheet.
2. Select the table containing nutrient values per 100 g.
3. Create the target variable from the first two digits of the classification code.
4. Remove rows where the target could not be created.
5. Remove broad groups with fewer than 50 samples.
6. Remove food name, classification code, derivation notes and other non-nutrient columns from the predictors.
7. Keep only numerical nutrient columns as input features.
8. Split the data into training and test sets using stratification, so the class proportions stayed similar in both sets.
9. Use median imputation for missing numerical values.
10. Apply standard scaling for models that are sensitive to the numerical scale of features.

3.1 Justification of Column Choices

I separated the removed columns into two types. The first type was leakage or label-related information. This included the food name, detailed food description and full classification code. These columns are related to

the answer, so including them would make the task too easy. For example, a food name can directly contain words such as milk, bread, beef or oil. If those words were used, the model might learn the food group from the label text instead of from the nutrient profile. This would not match my research question.

The second type was administrative or non-nutrient information. These columns may be useful for database management, but they do not describe nutrient composition. I removed them because the objective was specifically to test whether nutrient values can predict food group. Keeping them could make the interpretation less clear.

I did not manually drop nutrient columns just because I thought they looked less important. This is an important distinction. The manual removal step was only for leakage, identifiers and non-nutrient columns. After that, I kept the numerical nutrient features and used model-based feature importance as an algorithmic check of which features were useful. This makes the process less arbitrary than choosing nutrient columns only by personal judgement.

3.2 Justification of Missing Value Treatment

Median imputation was used for missing nutrient values. I chose the median rather than the mean because nutrient data can contain strong outliers. For example, some nutrients may be very high in only a small number of foods. The median is less affected by such extreme values. Missing values were not simply removed because deleting all rows with any missing nutrient would reduce the dataset too much and could remove useful samples.

3.3 Scaling and Train-Test Split

Scaling was used for k-nearest neighbours and logistic regression. These methods are affected by feature scale. If one nutrient has values in thousands and another nutrient has values close to zero, the larger-scale feature can dominate the model even if it is not actually more important. Standardisation helps put features on a more comparable scale. Tree-based models, such as decision trees and random forests, do not need scaling in the same way, but I still used a consistent preprocessing pipeline where appropriate.

The data was split into a training set and a test set. The test set was only used for final evaluation after model comparison. Cross-validation on the training data was used to estimate model performance before checking the test set. This helps reduce the chance of choosing a model only because it happened to do well on one split.

4 Exploratory Data Analysis

Figure 1 shows the distribution of the selected broad food groups after filtering out small classes. The classes are clearly not perfectly balanced. The largest group has 355 samples, while the smallest selected groups have 56 or 57 samples. This matters because a model could get a high normal accuracy by doing well on the large classes while still performing badly on the small classes.

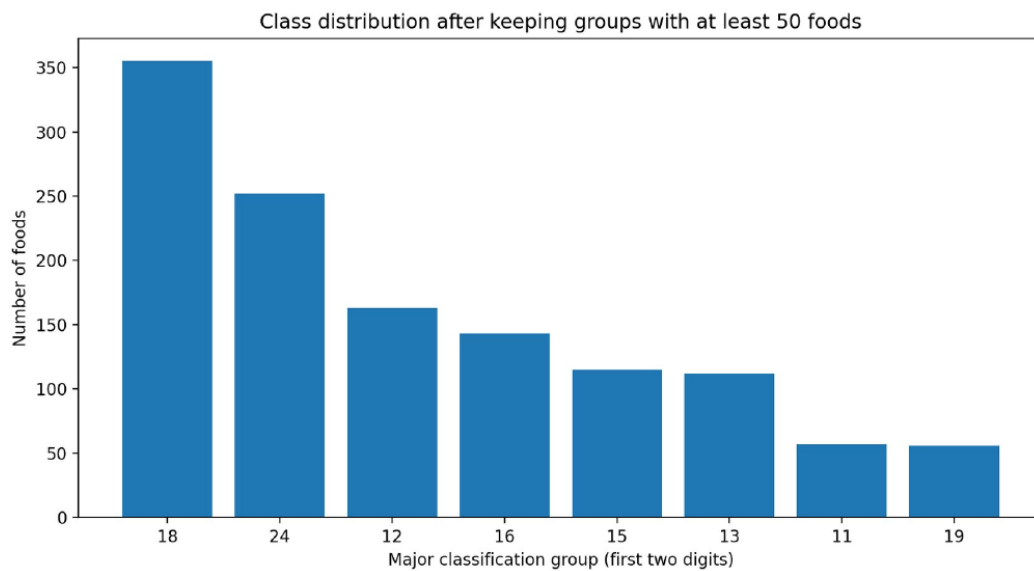


Figure 1: Distribution of selected broad food groups after filtering small classes.

Figure 2 shows the proportion of missing values among selected nutrient features. Some nutrient columns have more missing values than others. This is expected because not every nutrient is measured or reported for every food item. The missingness plot supports the decision to use imputation rather than assuming the spreadsheet is already clean.

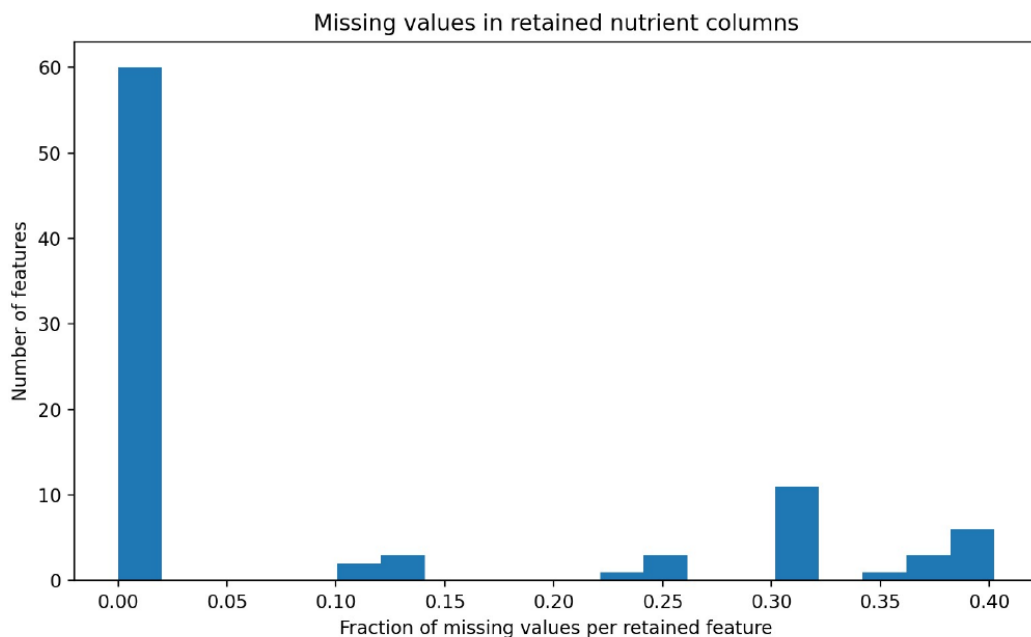


Figure 2: Missing values in nutrient features.

From the class distribution and missingness plots, there are two clear issues to consider. First, the dataset has class imbalance, so evaluation should not only use ordinary accuracy. Second, there are missing values, so preprocessing must be included as part of the method. These issues are common in real tabular datasets and are part of the machine learning process, not just small technical details.

5 Feature Selection and PCA Consideration

Because the dataset contains many nutrient variables, I considered whether dimensionality reduction or feature selection should be used. I did not want to simply delete nutrient columns by hand, because that could be subjective. Instead, I used the random forest feature importance output as an algorithmic feature-selection check.

The feature importance results are reported later in Figure 5. The top features included dietary fibre, starch, fatty acid measurements, carbohydrate measurements, nitrogen, selenium, iron, protein and iodine. These features are nutritionally meaningful, so they give evidence that the model is relying on useful nutrient information rather than random columns. This was also a practical feature selection step: it showed which variables would be most reasonable to keep if I wanted to build a smaller model.

I decided not to reduce the final model to only the top few features. The reason is that some food groups may be identified by combinations of many weaker nutrient signals. If I set an arbitrary cut-off such as top 10 or top 15 only, I might remove features that are individually weak but useful together. Since the full feature set contained 90 numerical nutrient variables, it was still small enough for the models used in this report.

I also considered PCA. PCA could reduce the feature space and remove some correlation between nutrient variables. However, I did not use PCA for the final reported models for three reasons. First, the main dataset size was manageable, so dimensionality reduction was not necessary for computational reasons. Second, PCA components are harder to interpret because each component is a mixture of many nutrients. This would make it harder to explain why the model works. Third, the best model, random forest, can already handle non-linear feature interactions and does not require uncorrelated input features in the same way as some other methods. Therefore, I kept the original nutrient variables so that the results and feature importance stayed more understandable.

6 Models

6.1 Dummy Baseline

The dummy classifier predicts the most frequent class. It does not learn a real relationship between nutrient features and food groups. I included it as a baseline because it gives a minimum reference point. If the real models only perform a little better than the dummy model, then the nutrient features may not be very useful or the method may not be suitable.

6.2 k-Nearest Neighbours

The k-nearest neighbours model predicts the class of a food item by looking at the most similar examples in the training data. I used $k = 7$. kNN is easy to understand and can work well when similar samples are close in feature space. However, it can be sensitive to feature scaling and can become less effective when there are many features. This model was included as a simple non-linear method.

6.3 Logistic Regression

Logistic regression is a linear classification model. For this multi-class task, it learns decision boundaries between different food groups. It is usually easier to interpret than more complex models. I included it to test whether the broad food groups can be separated using mostly linear patterns in nutrient values.

6.4 Decision Tree

A decision tree makes predictions by repeatedly splitting the dataset based on feature values. It is useful because it can model non-linear relationships and is more interpretable than many other models. However, a single decision tree can overfit if it grows too complex. It may learn details of the training data that do not generalise well.

6.5 Random Forest

A random forest is an ensemble of decision trees. Each tree is trained with some randomness, and the final prediction is based on the combined output of many trees. This often reduces overfitting compared with a single tree. Random forests are also good for tabular data and can provide feature importance values. I expected this model to perform well because nutrient relationships may be non-linear and may involve interactions between features.

7 Evaluation Metrics

I used several evaluation metrics: accuracy, balanced accuracy and macro F1 score. The most important metric in this report is balanced accuracy.

Ordinary accuracy is the percentage of correct predictions. It is easy to understand, but it can be misleading when classes are imbalanced. If one class is much larger than the others, a model can get a reasonable accuracy by mainly predicting that large class.

Balanced accuracy calculates the average recall across classes. This gives each class more equal importance, even when some classes are smaller. Since my selected food groups are not equally represented, balanced accuracy is more appropriate than accuracy alone.

Macro F1 score was also used. It averages the F1 score across classes. This gives another view of whether the model performs well across the different food groups rather than only the majority groups.

For cross-validation, I used the training data. The final test results were reported separately. This is important because the test set should represent unseen data. If I used the test set repeatedly during model selection, the final performance estimate would be less reliable.

8 Results

Table 1 summarises the model comparison. The dummy baseline performed very poorly, with a test balanced accuracy of 0.125. This is expected because there are 8 classes, and the dummy model does not learn useful patterns.

Model	CV balanced accuracy	Test accuracy	Test balanced accuracy
Dummy most frequent	0.125	0.283	0.125
kNN (k=7)	0.820	0.889	0.837
Logistic regression	0.882	0.927	0.918
Decision tree	0.845	0.924	0.903
Random forest	0.906	0.981	0.974

Table 1: Model comparison using cross-validation and test-set performance.

The kNN model performed much better than the baseline. Its test balanced accuracy was 0.837. This suggests that food items in the same broad group often have similar nutrient profiles. However, kNN was weaker than logistic regression and random forest. One possible reason is that the dataset has many nutrient features, and distance-based models can struggle when there are many dimensions.

Logistic regression performed surprisingly well. Its test balanced accuracy was 0.918 and its test accuracy was 0.927. This suggests that many food groups can be separated using relatively simple linear combinations of nutrient values. This is useful because it shows that the task is not only solved by a very complex model.

The single decision tree achieved a test balanced accuracy of 0.903. This is strong, but lower than logistic regression and random forest. A possible explanation is that a single tree may overfit some training patterns or make unstable splits. The tree can capture non-linear structure, but it may not generalise as smoothly as an ensemble model.

The random forest was the best model overall. It achieved a test accuracy of 0.981, a test balanced accuracy of 0.974 and a test macro F1 score of 0.976. This means it performed well not only on the majority classes but also across the different selected food groups. Figure 3 shows the model comparison visually.

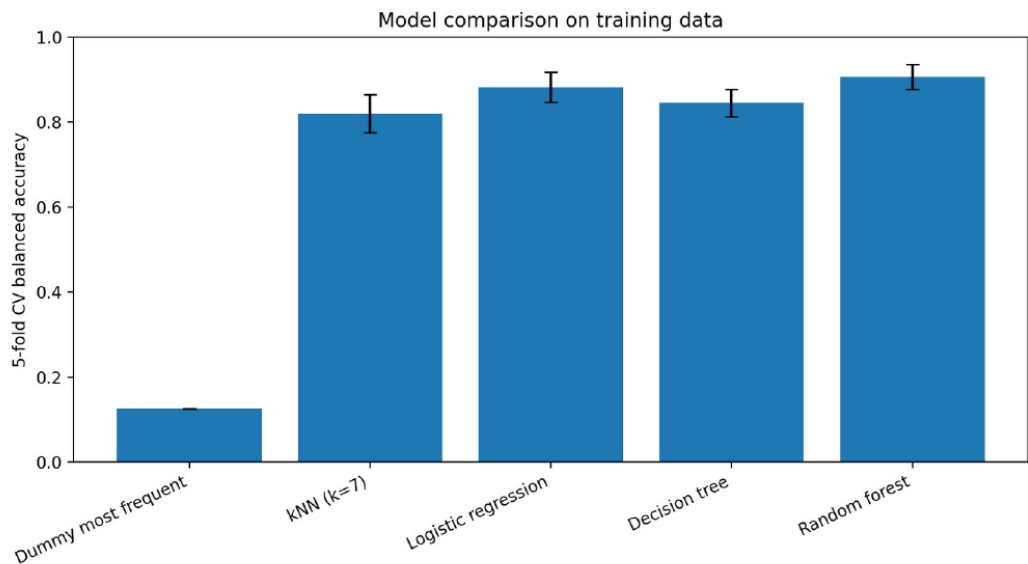


Figure 3: Comparison of model performance across the selected models.

9 Random Forest Confusion Matrix

The confusion matrix for the random forest is shown in Figure 4. Most predictions are on the diagonal, which means the model correctly classified most test samples. The classification report also showed high precision, recall and F1 scores for most groups. For example, groups 18 and 19 were predicted with perfect or near-perfect performance in the test set, while groups such as 12 and 13 had slightly lower but still strong performance.

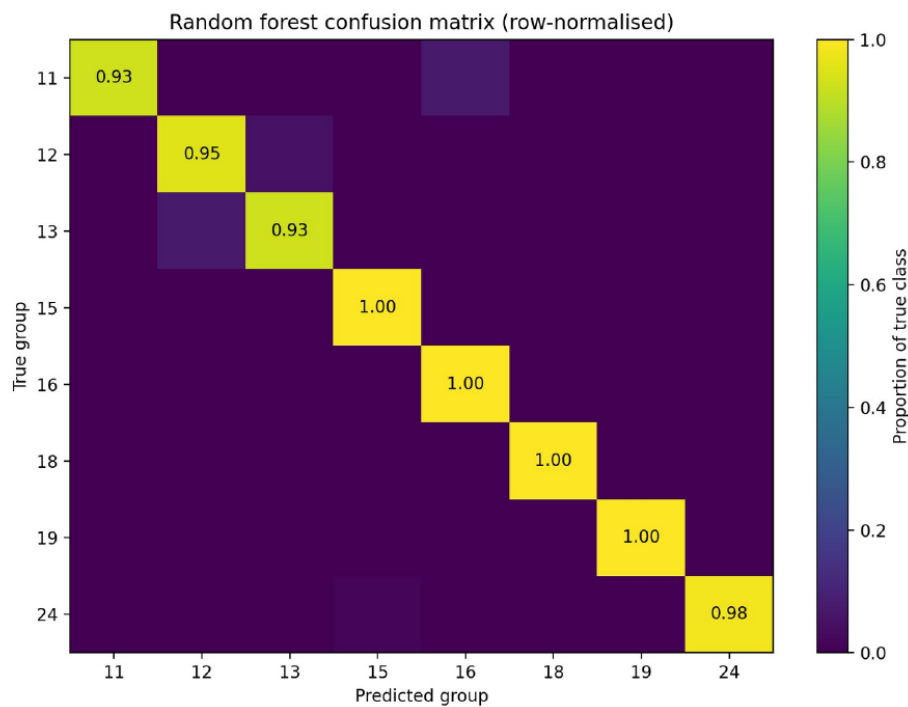


Figure 4: Random forest confusion matrix on the test set.

The small number of mistakes is still important. They probably occur between food groups with similar nutrient profiles. For example, some processed or mixed foods may not have a very clear nutrient identity. Also, some smaller groups have fewer test examples, so one mistake can make their recall look worse. This is why balanced accuracy and the confusion matrix are more informative than only reporting accuracy.

The strong random forest result suggests that broad food group information is strongly encoded in nutrient composition. This is sensible because food classification groups are not random labels. They are related to real food types, and food types often have different nutrient patterns.

10 Feature Importance

Figure 5 shows the top random forest feature importance values. The most important features included total dietary fibre, starch, C22:6w3, saturated fatty acids, available carbohydrate, long chain omega 3 fatty acids, nitrogen, selenium, monounsaturated fatty acids, iron, protein, trans fatty acids and iodine.

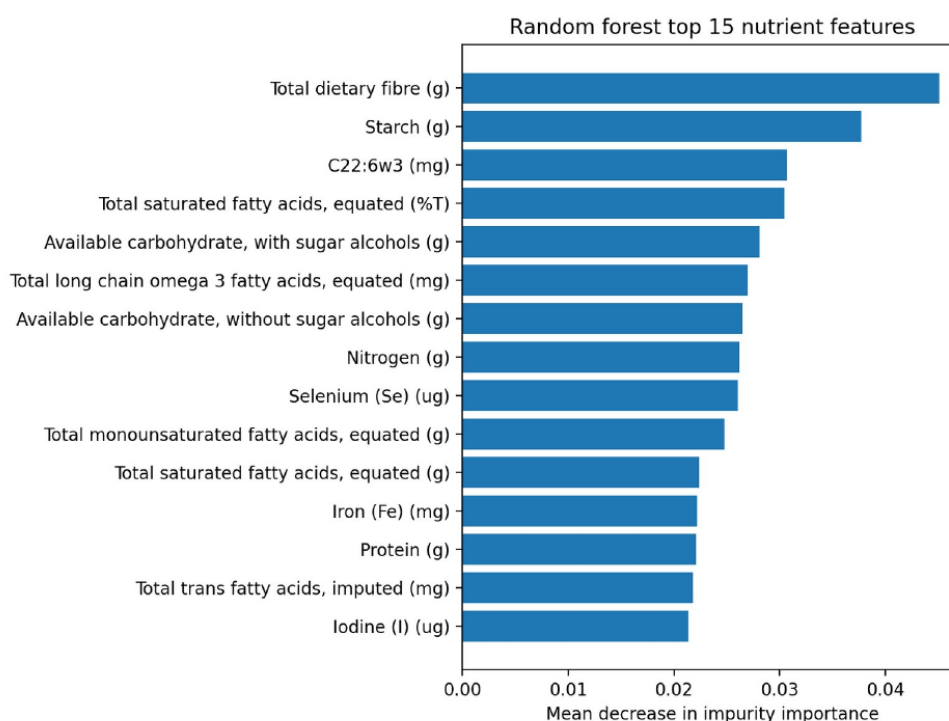


Figure 5: Random forest feature importance values.

Feature	Random forest importance
Total dietary fibre (g)	0.045
Starch (g)	0.038
C22:6w3 (mg)	0.031
Total saturated fatty acids, equated (%T)	0.030
Available carbohydrate, with sugar alcohols (g)	0.028
Total long chain omega 3 fatty acids, equated (mg)	0.027
Available carbohydrate, without sugar alcohols (g)	0.026
Nitrogen (g)	0.026
Selenium (Se) (ug)	0.026
Total monounsaturated fatty acids, equated (g)	0.025

Table 2: Top ten random forest feature importance values.

These features make sense for the problem. Starch, carbohydrate and fibre can help identify cereal-based or plant-based foods. Protein and nitrogen can help separate higher-protein groups. Fatty acid features can help distinguish foods with different fat compositions. Minerals and trace elements may also help identify food groups such as seafood, meat, grains or other specific categories.

However, feature importance should be interpreted carefully. A high importance value does not prove that a nutrient causes a food to be in a certain group. It only means the feature helped the model make predictions. Also, if two features are correlated, the importance may be shared between them or one may appear more important than the other. For example, protein and nitrogen are naturally related because nitrogen is used in protein estimation. Similarly, different carbohydrate measures may overlap.

Even with these limitations, feature importance makes the random forest result more understandable. It shows that the model is using nutritionally meaningful information rather than producing a high score for no obvious reason.

11 Discussion

The experiment shows that nutrient profiles can be used to predict broad food classification groups very effectively. All real machine learning models performed much better than the dummy baseline. This confirms that the nutrient features contain useful information about food group labels.

The comparison between models is also informative. kNN performed well, which means similar foods are often close to each other in nutrient space. Logistic regression performed even better, which suggests that many broad food groups are separable by relatively simple decision boundaries. The decision tree also performed strongly but did not beat logistic regression. The random forest performed best because it can capture non-linear relationships and reduce the instability of a single tree.

The very high random forest performance should not be interpreted as a surprising or magical result. The target variable is a food classification group, and food classification is naturally related to nutrient composition. Therefore, the task is easier than predicting something less directly related, such as a person's preference or a hidden property that is not reflected in the nutrient values. A high score here mainly means that the dataset has a strong relationship between features and labels.

There are also limitations in my experiment. First, I filtered out food groups with fewer than 50 samples. This made the model evaluation more stable, but it also means the final model was not tested on rare groups. In a real application, rare groups might still be important. Second, I predicted only broad food groups, not detailed categories. More detailed categories would probably be harder to predict and may reveal more confusion between similar foods. Third, I used median imputation for missing values. This is a reasonable simple method, but missingness itself may contain information. A future experiment could add missing-value indicators or compare different imputation methods.

Another limitation is that I only used a moderate number of models and did not do a full hyperparameter search. The assignment is open-ended, and it would be possible to do more tuning. For example, the random forest could be tuned for the number of trees, maximum depth and minimum samples per leaf. Logistic regression could be tuned using different regularisation strengths. A neural network could also be tried, but I did not use it as the main model because the tree-based models already gave strong results and were easier to interpret for this tabular dataset.

The feature-selection decision is also a limitation. I used random forest feature importance to check the most useful variables and to make the feature discussion less arbitrary. However, I did not train a final reduced-feature model. A future improvement would be to compare the full random forest with a model using only the top 10, top 20 or top 30 features. That would test whether the high performance depends on many nutrient variables or only a small set.

PCA is another possible extension. PCA might help if many nutrient variables are strongly correlated. However, I chose not to use it in the final model because this report focuses on understanding the relationship between nutrients and food groups. PCA would create components that are harder to explain in normal nutrient terms. For this assignment, keeping interpretability was more valuable than reducing the number of variables.

I also think it is important that food names were not used as predictors. If names were used, the model might learn text clues instead of nutrient patterns. Removing names made the experiment more honest. It meant that the model had to classify foods based on numerical nutrient information, which matched my research question.

12 Assumptions

I made the following assumptions in this analysis:

- The first two digits of the classification code are a reasonable representation of a major food group.
- Groups with fewer than 50 rows are too small for stable evaluation in this report, so they were removed.

- Food names and administrative columns should not be used as predictors because they could leak direct label information.
- Median imputation is an acceptable simple method for missing nutrient values because nutrient values can have outliers.
- Balanced accuracy is more suitable than accuracy alone because the selected classes are imbalanced.
- Random forest feature importance is a reasonable way to check which retained nutrient variables are useful, but it should not be interpreted as causal evidence.
- PCA was considered, but interpretability was prioritised over dimensionality reduction.

These assumptions affect the scope of the results. The conclusion applies to the selected broad food groups and the selected nutrient features. It should not be treated as proof that all possible food categories can be predicted perfectly from all nutrient datasets.

13 Reflection

Before doing this assignment, I mostly thought machine learning was about choosing a model and checking the accuracy. After doing the experiment, I understand more clearly that the choices before training the model are just as important. I had to decide what the target should be, which columns should be used, how to handle missing values, and what metric is fair for the problem.

At the beginning I was quite confused because the assignment was very open. I was not sure whether I should try many models or focus on explaining a few models properly. Later I realised that the marks are not only about the highest score, but also about explaining why I made each choice. This made me slow down and think more carefully about the steps.

One thing I learned is that a high accuracy number is not enough by itself. At first, I wanted to make the model score as high as possible. But when I looked at the class distribution, I realised that accuracy can be misleading when the classes are not equal. Balanced accuracy and the confusion matrix helped me understand the result better. This connected to the course idea that evaluation should match the problem, not just use one default number.

I also learned more about data leakage. If I used food names as features, the model might predict the food group from words rather than from nutrients. That would give a strong score but would not really answer the question. Removing food names made the task more fair. This was one of the most important decisions in the assignment.

Another thing I noticed is that simpler models can still be useful. I expected random forest to do well, but logistic regression also performed strongly. This helped me understand that it is useful to compare models instead of only using the most complex one. A simpler model can show whether the data already has clear patterns.

I also learned that I need to justify choices, not only list the steps. For example, saying I removed columns is not enough. I need to explain whether they were removed because they were leakage, because they were not nutrient values, or because they were not useful for the research question. I also learned that feature selection should not just be based on my personal feeling. Looking at random forest feature importance helped me make this part more systematic.

The PCA decision was also useful for my understanding. Before this assignment, I thought PCA was always a good thing when there are many features. Now I understand that PCA has a trade-off. It can reduce dimensions, but it can also make the model harder to explain. Since this report needed discussion and interpretation, I decided the original nutrient features were better for the final analysis.

The hardest part for me was deciding the scope. The assignment is very flexible, so I was not sure how much was enough. I learned that it is better to do a smaller number of experiments properly and explain them

clearly, instead of trying too many models without enough discussion. This assignment helped me connect practical work with lecture ideas such as classification, preprocessing, train-test splitting, overfitting, feature scaling, class imbalance and model interpretation.

14 Conclusion

This report investigated whether numerical nutrient profiles can predict broad food classification groups. I treated the task as a multi-class classification problem and compared a dummy baseline, kNN, logistic regression, a decision tree and a random forest model.

The results show that the nutrient profiles are highly informative for this task. The dummy baseline performed poorly, while all real models performed much better. The random forest classifier gave the best result, with a test balanced accuracy of 0.974 and a macro F1 score of 0.976. The confusion matrix showed that most food groups were predicted correctly, and the feature importance plot showed that meaningful nutrient features were used by the model.

Overall, the experiment demonstrates a complete machine learning workflow on a real dataset. It also shows that model evaluation and interpretation are important. The best result is not only the model with the highest score, but the model whose behaviour can be explained and connected back to the data and the problem.

AI Assistance Statement

AI assistance was used for wording support, report organisation, and checking whether the explanation was clear. The dataset choices, modelling decisions, interpretation of results, and final edits were reviewed by me.

References

Food Standards Australia New Zealand. Australian Food Composition Database. Available at: <https://www.foodstandards.gov.au/science-data/food-nutrient-databases/afcd>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830.

McKinney, W. (2010). Data Structures for Statistical Computing in Python. In *Proceedings of the 9th Python in Science Conference*.

Appendix A: Main Python Workflow Summary

The following code outline shows the main workflow used for the experiment. The code was used to load the spreadsheet, select numerical nutrient features, create the broad classification target, impute missing values, scale features when needed, train models, compute metrics, save plots and extract random forest feature importance. I include a workflow summary rather than every line of plotting code so the appendix stays readable.

```
# Outline only
X = nutrient_numeric_features
y = major_food_group

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

numeric_preprocess = SimpleImputer(strategy="median")
scaled_preprocess = Pipeline([
```

```

        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])

models = {
    "Dummy most frequent": DummyClassifier(strategy="most_frequent"),
    "kNN (k=7)": Pipeline([
        ("prep", scaled_preprocess),
        ("model", KNeighborsClassifier(n_neighbors=7))
    ]),
    "Logistic regression": Pipeline([
        ("prep", scaled_preprocess),
        ("model", LogisticRegression(max_iter=5000))
    ]),
    "Decision tree": Pipeline([
        ("prep", numeric_preprocess),
        ("model", DecisionTreeClassifier(random_state=42))
    ]),
    "Random forest": Pipeline([
        ("prep", numeric_preprocess),
        ("model", RandomForestClassifier(
            n_estimators=300,
            random_state=42,
            class_weight="balanced"
        ))
    ])
}

for name, model in models.items():
    cv_scores = cross_validate(
        model, X_train, y_train, cv=5,
        scoring={
            "accuracy": "accuracy",
            "balanced_accuracy": "balanced_accuracy",
            "macro_f1": "f1_macro"
        }
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(name)
    print(balanced_accuracy_score(y_test, y_pred))
    print(f1_score(y_test, y_pred, average="macro"))

best_model = models["Random forest"]
best_model.fit(X_train, y_train)
rf = best_model.named_steps["model"]
feature_importance = rf.feature_importances_

```